# 21.1 Points and Boxes

A *point* $\mathbf{p}$ in a $D$-dimensional space is specified by its $D$ Cartesian coordinates, $(x_0, x_1, \ldots, x_{D-1})$. Generally we will concern ourselves only with the cases $D = 2$ (points in a plane) and $D = 3$ (points in 3-space), but the concept is more general.

The representation in code follows just this paradigm. By eschewing special names for individual coordinates — like $x$, $y$, $z$ — we keep the ability to loop easily over coordinates in $D$ dimensions.

pointbox.h
```
template<Int DIM> struct Point {
Simple structure to represent a point in DIM dimensions.
    Doub x[DIM];                                          The coordinates.
    Point(const Point &p) {                               Copy constructor.
        for (Int i=0; i<DIM; i++) x[i] = p.x[i];
    }
    Point& operator= (const Point &p) {                   Assignment operator.
        for (Int i=0; i<DIM; i++) x[i] = p.x[i];
        return *this;
    }
    bool operator== (const Point &p) const {
        for (Int i=0; i<DIM; i++) if (x[i] != p.x[i]) return false;
        return true;
    }
    Point(Doub x0 = 0.0, Doub x1 = 0.0, Doub x2 = 0.0) {
        x[0] = x0;                                        Constructor by coordinate values. Arguments
        if (DIM > 1) x[1] = x1;                             beyond the required number are not used
        if (DIM > 2) x[2] = x2;                             and can be omitted.
        if (DIM > 3) throw("Point not implemented for DIM > 3");
    }
};
```

In the interest of concise code, the constructor above may pass some unnecessary default arguments of zero. You can easily clean this up if you care.

If we have two points $\mathbf{p}$ and $\mathbf{q}$, we can compute their distance $d$,

$$d = |\mathbf{p} - \mathbf{q}| = \left[ \sum_{i=0}^{D-1} (p_i - q_i)^2 \right]^{1/2} \tag{21.1.1}$$

where $p_i$ and $q_i$ are now the respective Cartesian coordinates for each point.

In code, we have

pointbox.h
```
template<Int DIM> Doub dist(const Point<DIM> &p, const Point<DIM> &q) {
Returns the distance between two points in DIM dimensions.
    Doub dd = 0.0;
    for (Int j=0; j<DIM; j++) dd += SQR(q.x[j]-p.x[j]);
    return sqrt(dd);
}
```

Note that `dist` is not a member of the class `Point`, but rather a freestanding function whose arguments are `Points`. We will overload `dist` with other types of arguments, signifying other kinds of distances between objects.

## 21.1.1 Boxes

By a *box*, we mean a rectangle (for $D = 2$) or rectangular parallelepiped (for $D = 3$, a "brick" in other words) that is aligned with the coordinate axes. Boxes are

interesting because they can tessellate (that is, partition) $D$-dimensional space, and they can contain other objects. Indeed, every finite, extended object has a *bounding box*, which is the unique smallest box that contains it. One way to represent a box is by the points at two special, diagonally opposite, corners. The first point ("low") has coordinate values that are the minima on the surface of the box; the second point ("high") has coordinate values that are the maxima. All other corners of a box, it should be obvious, have coordinate values that are, dimension by dimension, either the value of "low" or the value of "high"; and all such permutations are corners, $2^D$ in all.

The code follows this description:

pointbox.h
```
template<Int DIM> struct Box {
```
Structure to represent a Cartesian box in DIM dimensions.
```
    Point<DIM> lo, hi;
    Box() {}
    Box(const Point<DIM> &mylo, const Point<DIM> &myhi) : lo(mylo), hi(myhi) {}
};
```
Diagonally opposite corners (min of all coordinates and max of all coordinates) are stored as two points.

Note that a copy constructor and assignment operator are not needed, since by default the two `Points` will be appropriately copied or assigned (one convenience of this representation).

A point can be either outside a box, inside it, or — in principle — on its surface. As mentioned in §21.0, we represent all coordinates as (approximate) floating-point numbers, not (exact) integers, so it would not be prudent to depend on any exact equalities of coordinate values or distances. We will be careful, therefore, not to put too much credence in the idea of the exact surface of a box; usually we'll consider the surface (should some exact equality happen to hold) as a part of the box's interior.

If a point is outside a box, then we define its distance from the box to be the distance to the nearest point on the surface of (or inside) the box. A glance at Figure 21.1.1 shows that this distance is the Pythagorean sum (that is, square root of sum of squares) of the distances from the point to some — but not all — of the hyperplanes that bound the box. The rule is that when a point has a coordinate that is greater than the corresponding max of the box, or less than the corresponding min, then *that* coordinate contributes to the sum. When the point has a coordinate between the max and min, then it does *not* contribute to the sum, since (along that coordinate) the shortest line can be perpendicular to the hyperplane. When a point is inside, or on the surface of, a box, we define its distance to the box to be zero.

These definitions of distance are embodied in the following code.

pointbox.h
```
template<Int DIM> Doub dist(const Box<DIM> &b, const Point<DIM> &p) {
```
If point p lies outside box b, the distance to the nearest point on b is returned. If p is inside b or on its surface, zero is returned.
```
    Doub dd = 0;
    for (Int i=0; i<DIM; i++) {
        if (p.x[i]<b.lo.x[i]) dd += SQR(p.x[i]-b.lo.x[i]);
        if (p.x[i]>b.hi.x[i]) dd += SQR(p.x[i]-b.hi.x[i]);
    }
    return sqrt(dd);
}
```

Frequently we want to know if a point is inside or outside a box. The above `dist` routine can be used for this. A positive return means outside, zero means